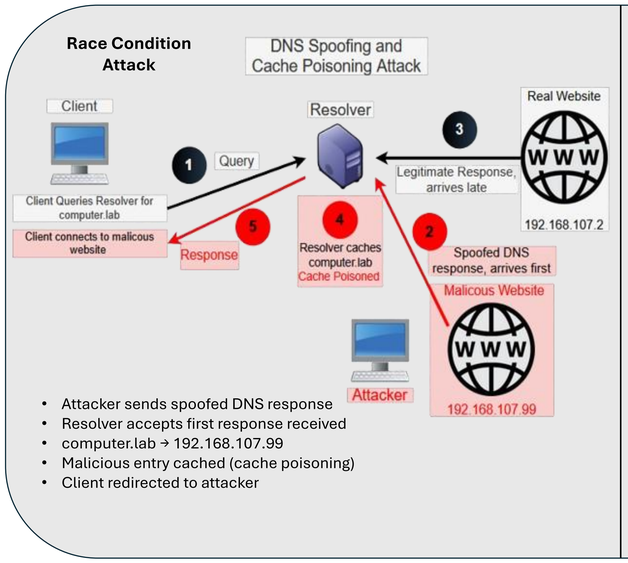
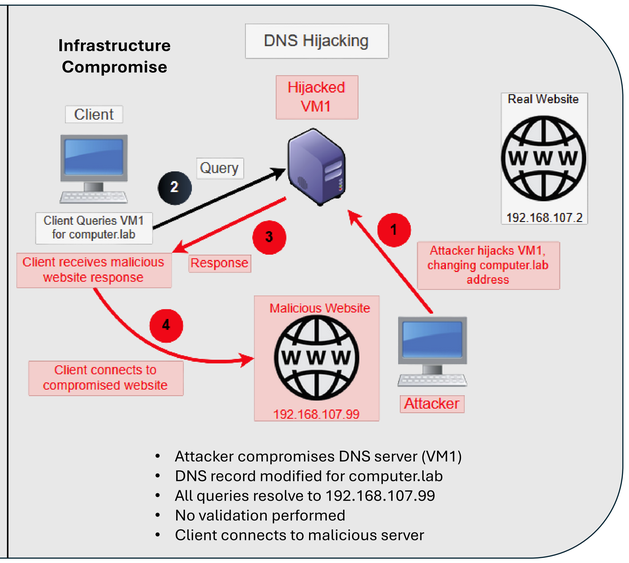
DNSSEC: A Study Into Manual Signing and Validation of Signatures

Steven Dormady
Advisor: Mohamed Aboutabl Ph.D

Motivation – DNS Vulnerabilities and Attacks

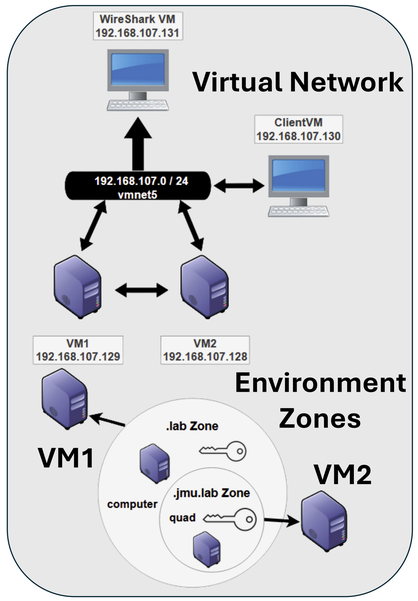


- Attacker sends spoofed DNS response
- Resolver accepts first response received
- computer.lab → 192.168.107.99
- Malicious entry cached (cache poisoning)
- Client redirected to attacker



- Attacker compromises DNS server (VM1)
- DNS record modified for computer.lab
- All queries resolve to 192.168.107.99
- No validation performed
- Client connects to malicious server

Testbed Environment



Simulated Attack - No DNSSEC

DNS Hijacking

```
$TTL    86400
lab.    IN      SOA     ns.lab. root.lab. (
                                15          ; Serial
                             43200          ; Refresh
                               900          ; Retry
                           1814400          ; Expire
                              7200 )        ; minimum

lab.    IN      NS      ns.lab.
ns      IN      A       192.168.107.129

jmu.lab.        IN      NS      ns.evil.lab.
ns.evil.lab.    IN      A       192.168.107.200

bc      IN      A       192.168.107.1
computer        IN      A       192.168.107.2

jmu.lab. IN DS 34939 8 2 968205142 2A73107F882F98D04
```

Originally ns.jmu.lab → 192.168.107.128

- Attacker modifies DNS record on VM1
- ns.evil.lab → 192.168.107.200
- Resolver returns malicious IP
- Client redirected to attacker-controlled server

```
steve@steve-VMware-Virtual-Platform:~$ dig @192.168.107.129 jmu.lab. NS

; <<>> DiG 9.18.39-0ubuntu0.24.04.2-Ubuntu <<>> @192.168.107.129 jmu.la
rec
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 52499
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: 43a03a13c7ad7e9a0100000069b49377d30f4e3f16062eae (good)
;; QUESTION SECTION:
;jmu.lab.                       IN      NS

;; AUTHORITY SECTION:
jmu.lab.                86400   IN      NS      ns.evil.lab.
```

Above is a query without DNSSEC, and the redirect is successful to the malicious name server

Simulated Attack: DNSSEC Enabled

Validated Query

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key +
.107.129 computer.lab A
; fully validated
computer.lab.           86400   IN      A       192.168.107.2
computer.lab.           86400   IN      RRSIG   A 8 2 86400 20
31323532 34104 lab. NWPQcfaC2Th/jhCeSP/QlhLYrXB9+CZJ8azBH6bft
smskBIMSnGCeMUTgX+jnS4ET5Rp7FJbOdefkTi9B4fvUGSt0rccvEB zTwS+NV
eq8bklL6ImYI4b02kypF2wxYyD4wFXU JWA=
```

- This terminal capture shows a successful, fully validated *delv* output.

Validation Failed – Tampered Data Rejected

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key
.107.129 jmu.lab NS +rtrace
;; fetch: jmu.lab/NS
;; DNS format error from 192.168.107.129#53 resolving jmu.lal
non-improving referral
;; FORMERR resolving 'jmu.lab/NS/IN': 192.168.107.129#53
; resolution failed: failure
```

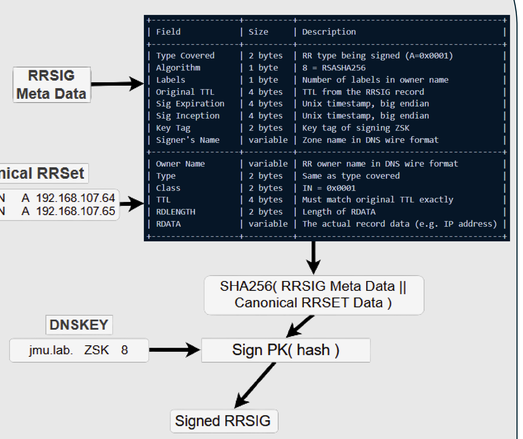
The parent server is being queried for the name server of jmu.lab. However, the zone is currently set up for ns.evil.lab, meaning the real Name Server will not be returned or validated.

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key
.107.129 quad.jmu.lab A
;; DNS format error from 192.168.107.129#53 resolving quad.jr
n>: non-improving referral
;; FORMERR resolving 'quad.jmu.lab/A/IN': 192.168.107.129#53
; resolution failed: failure
```

Here, I am trying to query the parent about the child, but the child is no longer reachable. This fails and claims it is "unknown".

Resource Record Signature Data

Composition of a RRSIG



Field	Size	Description
Type Covered	2 bytes	RR type being signed (A=0x0001)
Algorithm	1 byte	8 = RSASHA256
Labels	1 byte	Number of labels in owner name
Original TTL	4 bytes	TTL from the RRSIG record
Sig Expiration	4 bytes	Unix timestamp, big endian
Sig Inception	4 bytes	Unix timestamp, big endian
Key Tag	2 bytes	Key tag of signing ZSK
Signer's Name	variable	Zone name in DNS wire format
Owner Name	variable	RR owner name in DNS wire format
Type	2 bytes	Same as type covered
Class	2 bytes	IN = 0x0001
TTL	4 bytes	Must match original TTL exactly
RDLENGTH	2 bytes	Length of RDATA
RDATA	variable	The actual record data (e.g. IP address)

This process replicates DNSSEC validation by manually reconstructing the exact byte sequence required for signature verification, demonstrating how DNSSEC prevents tampered DNS responses.

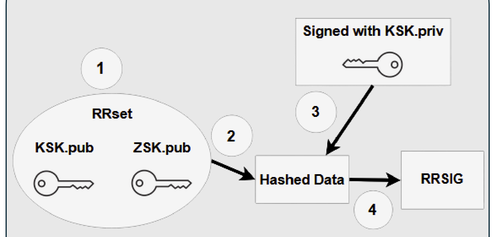
Canonical Resource Record Set

```
king.jmu.lab.   3600   IN   A  192.168.107.60
quad.jmu.lab.   3600   IN   A  192.168.107.64
quad.jmu.lab.   3600   IN   A  192.168.107.65
```

A canonical RRSet is a standardized way of ordering records for consistency in building signatures across DNSSEC. The order of names is alphabetical, if names are the same in the RRSet, then focus on RDATA (Address, Name Server, etc.).

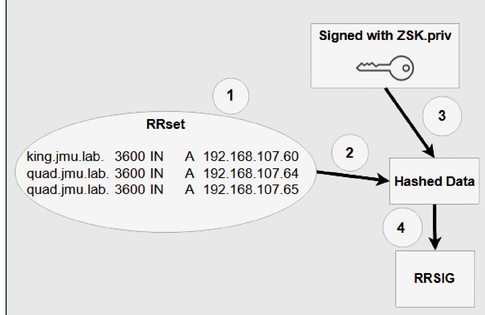
Signing and Query Process

DNSKEY Query



1. The Record is assembled in canonical order, appended to the RRSIG meta data.
2. This data is then hashed with the specified algorithm.
3. The Key-Signing Key is used to sign the hashed data.
4. The signature is then stored, appended to the key data, then sent out when queried.

Address Record Query



1. The Record is assembled in canonical order, appended to the RRSIG meta data.
2. This data is then hashed with the algorithm specified.
3. The private Zone-Signing Key signs the hashed data.
4. The signature is appended to the RRSET and stored, then sent out when queried.

Corrupted Signatures

Corrupted Key

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key +root=lab
.107.129 computer.lab A
;; validating lab/DNSKEY: no valid signature found (DS)
;; no valid RRSIG resolving 'lab/DNSKEY/IN': 192.168.107.129#53
;; broken trust chain resolving 'computer.lab/A/IN': 192.168.107.129#5
;; resolution failed: broken trust chain
```

- Trust anchor used for validation
- Key modified (single character change)
- Chain of trust broken, trust anchor corrupted
- Validation fails before signature check

Corrupted Signature

```
steve@steve-VMware-Virtual-Platform:~$ delv -a lab-trust.key
; fetch: computer.lab/A
; fetch: lab/DNSKEY
; validating computer.lab/A: no valid signature found
; RRSIG failed to verify resolving 'computer.lab/A/IN': 192
; resolution failed: RRSIG failed to verify
```

- Valid key, but signature modified
- RRSIG no longer matches RRSET
- Cryptographic verification fails
- Response rejected

These tests demonstrate that DNSSEC detects even minor data tampering and prevents acceptance of invalid responses.

Final Takeaways

Without DNSSEC:
- ✗ DNS responses can be spoofed or manipulated

With DNSSEC:
- ✓ Responses are cryptographically verified
- ✓ Tampered data is rejected
- ✓ Attacks fail
- Integrity and Authentication are ensured

Manual Signing and Verification

Signature Generation Code

```
// --- Generate the signature ---
unsigned char *gen_sig = NULL;
size_t gen_sig_len = 0;

EVP_MD_CTX *sign_ctx = EVP_MD_CTX_new();
if (EVP_DigestSignInit(sign_ctx, NULL, EVP_sha256(), NULL, privkey) != 1 ||
    EVP_DigestSignUpdate(sign_ctx, sigbase, sigbase_len) != 1 ||
    EVP_DigestSignFinal(sign_ctx, NULL, &gen_sig_len) != 1) {
    fprintf(stderr, "Error: signing init failed\n");
    return 1;
}

gen_sig = malloc(gen_sig_len);

if (EVP_DigestSignFinal(sign_ctx, gen_sig, &gen_sig_len) != 1) {
    fprintf(stderr, "Error: signing final failed\n");
    return 1;
}
```

The above code shows the steps that go into hashing the RRSIG and RRSet Data, then signing it with the respective private key, the ZSK for zone data or the KSK for keys.

Signature Validation Code

```
// --- Verify our signature with public key ---
printf("=== Verify: generated signature with public key ===\n");
EVP_MD_CTX *verify_ctx = EVP_MD_CTX_new();
if (EVP_DigestVerifyInit(verify_ctx, NULL, EVP_sha256(), NULL, pubkey) == 1 &&
    EVP_DigestVerifyUpdate(verify_ctx, sigbase, sigbase_len) == 1 &&
    EVP_DigestVerifyFinal(verify_ctx, gen_sig, gen_sig_len) == 1)
    printf("PASS: generated signature verifies correctly\n\n");
else
    printf("FAIL: generated signature did not verify\n\n");
EVP_MD_CTX_free(verify_ctx);

// --- Verify existing RRSIG with public key ---
printf("=== Verify: existing RRSIG with public key ===\n");
verify_ctx = EVP_MD_CTX_new();
if (EVP_DigestVerifyInit(verify_ctx, NULL, EVP_sha256(), NULL, pubkey) == 1 &&
    EVP_DigestVerifyUpdate(verify_ctx, sigbase, sigbase_len) == 1 &&
    EVP_DigestVerifyFinal(verify_ctx, existing_sig, existing_sig_len) == 1)
    printf("PASS: existing RRSIG verifies correctly\n\n");
else
    printf("FAIL: existing RRSIG did not verify - sigbase may not match exactly\n\n");
EVP_MD_CTX_free(verify_ctx);

// --- Binary comparison ---
compare_sigs(gen_sig, gen_sig_len, existing_sig, existing_sig_len);
```

The above code shows the steps that go into hashing the RRSIG and RRSet Data, then signing it with the respective private key, the ZSK for zone data or the KSK for keys.

Successful Signing - NS

```
=== Existing RRSIG signature (128 bytes) ===
0000  bb 2f 37 ae c3 2b c7 d6  68 1b 1f 4c 3d b1 06 a1  |./7..+..h..L=...|
0010  b0 02 61 08 11 94 c6 d2  1c 10 a1 99 c8 c3 5d 1a  |..a...........]. |
0020  5a 0b 4a 0d 5c cf 31 e3  9d b8 94 be 8e 41 94 d3  |Z.J.\.1......A..|
0030  89 15 61 eb 5b d8 86 2d  20 78 42 67 f9 ce 1a 2c  |..a.[..- xBg...,|
0040  50 de 50 b2 12 a3 4a 25  e2 a3 39 5a 77 b6 57 42  |P.P...J%..9zw.WB|
0050  c2 1a 49 22 ee 67 56 03  d9 89 40 be dd 51 2b e8  |..I".gV...@..Q+.|
0060  23 9d 2f 29 47 14 76 6d  0a a9 20 5a bb 8f 36 42  |#./)G.vm.. Z..6B|
0070  c4 29 01 7f 84 67 ee 43  71 af 12 c1 a0 bc 23 73  |.)...g.Cq.....#s|

=== Our generated signature (128 bytes) ===
0000  bb 2f 37 ae c3 2b c7 d6  68 1b 1f 4c 3d b1 06 a1  |./7..+..h..L=...|
0010  b0 02 61 08 11 94 c6 d2  1c 10 a1 99 c8 c3 5d 1a  |..a...........]. |
0020  5a 0b 4a 0d 5c cf 31 e3  9d b8 94 be 8e 41 94 d3  |Z.J.\.1......A..|
0030  89 15 61 eb 5b d8 86 2d  20 78 42 67 f9 ce 1a 2c  |..a.[..- xBg...,|
0040  50 de 50 b2 12 a3 4a 25  e2 a3 39 5a 77 b6 57 42  |P.P...J%..9zw.WB|
0050  c2 1a 49 22 ee 67 56 03  d9 89 40 be dd 51 2b e8  |..I".gV...@..Q+.|
0060  23 9d 2f 29 47 14 76 6d  0a a9 20 5a bb 8f 36 42  |#./)G.vm.. Z..6B|
0070  c4 29 01 7f 84 67 ee 43  71 af 12 c1 a0 bc 23 73  |.)...g.Cq.....#s|

=== Verify: our signature with public key ===
PASS: our signature verifies correctly

=== Verify: existing RRSIG with public key ===
PASS: existing RRSIG verifies correctly

=== Binary Comparison ===
Generated signature length : 128 bytes
Existing  signature length : 128 bytes
MATCH: signatures are byte-for-byte identical
```

Successful Signing - A

```
=== Existing RRSIG signature (128 bytes) ===
0000  63 bf 85 25 2a 93 a8 35  2b 1f 05 21 e0 fb 67 40  |c..%*..5+..!..g@|
0010  35 28 1f ec 8b 6c dc cd  69 aa 6c 76 70 20 88 d3  |5(...l..i.lvp ..|
0020  19 f7 a6 91 28 5f fe b7  29 72 36 a3 0a 45 03 9d  |....(_..)r6..E..|
0030  06 13 a1 74 2c 5b 69 6d  c1 ab a0 45 e3 7f 56 c6  |...t,[im...E..V.|
0040  83 ec dc b2 04 af 04 dd  37 6e 99 5d ec 72 22 2b  |........7n.].r"+|
0050  99 1d ac 3d 38 b8 0f 02  b5 82 c1 a5 db 85 c7 3b  |...=8..........;|
0060  46 2f 9a e7 cb 83 ba b2  d0 fd c9 4e 66 d2 97 13  |F/.........Nf...|
0070  56 d4 90 c7 13 f4 20 2c  77 13 cd 2e 39 d8 e1 2a  |V..... ,w...9..*|

=== Our generated signature (128 bytes) ===
0000  63 bf 85 25 2a 93 a8 35  2b 1f 05 21 e0 fb 67 40  |c..%*..5+..!..g@|
0010  35 28 1f ec 8b 6c dc cd  69 aa 6c 76 70 20 88 d3  |5(...l..i.lvp ..|
0020  19 f7 a6 91 28 5f fe b7  29 72 36 a3 0a 45 03 9d  |....(_..)r6..E..|
0030  06 13 a1 74 2c 5b 69 6d  c1 ab a0 45 e3 7f 56 c6  |...t,[im...E..V.|
0040  83 ec dc b2 04 af 04 dd  37 6e 99 5d ec 72 22 2b  |........7n.].r"+|
0050  99 1d ac 3d 38 b8 0f 02  b5 82 c1 a5 db 85 c7 3b  |...=8..........;|
0060  46 2f 9a e7 cb 83 ba b2  d0 fd c9 4e 66 d2 97 13  |F/.........Nf...|
0070  56 d4 90 c7 13 f4 20 2c  77 13 cd 2e 39 d8 e1 2a  |V..... ,w...9..*|

=== Verify: our signature with public key ===
PASS: our signature verifies correctly

=== Verify: existing RRSIG with public key ===
PASS: existing RRSIG verifies correctly

=== Binary Comparison ===
Generated signature length : 128 bytes
Existing  signature length : 128 bytes
MATCH: signatures are byte-for-byte identical
```

This terminal output shows the signed hexadecimal signature of the RRSIG from a query and the one generated by my code. There are two checks, using the public key on each signature, the compares the binary of each hash.